

Module 3a: Prepare Ground Truth And Simulation Datasets

This notebook turns local evaluation evidence into managed AgentCore Evaluation datasets.

You will learn how expected outputs, tool expectations, assertions, and simulation scenarios move from local JSON evidence into AgentCore managed datasets. The main concept is dataset lineage: every managed dataset version should be traceable back to the evaluation cases and assumptions that created it.

```
In [1]: import json
import os
import sys
from pathlib import Path

import boto3
import pandas as pd
from IPython.display import display

def _find_section_dir() -> Path:
    start = Path.cwd().resolve()
    candidates = [start, *start.parents]
    for candidate in candidates:
        if candidate.name == "03-production-deployment" and (candidate / "dataset_contract.py").exists():
            return candidate
        child = candidate / "03-production-deployment"
        if (child / "dataset_contract.py").exists():
            return child
    raise FileNotFoundError("Could not locate 03-production-deployment directory")

SECTION_DIR = _find_section_dir()
REPO_ROOT = SECTION_DIR.parent
os.chdir(SECTION_DIR)
for path in [SECTION_DIR, REPO_ROOT / "02-evaluation-baseline"]:
    if str(path) not in sys.path:
        sys.path.insert(0, str(path))

try:
    %store -r REGION
    print(f"Loaded REGION from previous module: {REGION}")
except Exception:
    session = boto3.Session()
    REGION = session.region_name or "us-west-2"
    print(f"Using default region: {REGION}")

sts = boto3.client("sts", region_name=REGION)
try:
    ACCOUNT_ID = sts.get_caller_identity()["Account"]
except Exception:
    ACCOUNT_ID = os.environ.get("AWS_ACCOUNT_ID", "000000000000")

print(f"Section directory: {SECTION_DIR}")
```

```
print(f"Region: {REGION}")
print(f"Account: {ACCOUNT_ID}")
```

Loaded REGION from previous module: us-east-1
Section directory: /workshop/03-production-deployment
Region: us-east-1
Account: 390402578846

Step 1: Load Quality Evidence

This cell loads the evaluation artifacts created by the baseline notebook.

Look for the run ID, selected slice, evaluator registry version, and threshold version. These fields explain where the dataset examples came from and why they were selected.

```
In [2]: from dataset_contract import (
        DATASET_MANIFEST_PATH,
        POSTDEPLOY_GROUND_TRUTH_PATH,
        POSTDEPLOY_SIMULATION_SCENARIOS_PATH,
        SIMULATED_SCHEMA,
        PREDEFINED_SCHEMA,
        assert_managed_dataset_ready,
        build_local_dataset_artifacts,
        dataset_client_preflight,
        managed_dataset_source,
        write_local_dataset_artifacts,
        build_dataset_manifest,
    )

artifacts = build_local_dataset_artifacts(region=REGION, account_id=ACCOUNT_ID)
section02_contract = artifacts["section02_contract"]
predefined_examples = artifacts["predefined_examples"]
simulation_examples = artifacts["simulation_examples"]
postdeploy_ground_truth = artifacts["postdeploy_ground_truth"]
postdeploy_simulation_scenarios = artifacts["postdeploy_simulation_scenarios"]
dataset_manifest = artifacts["dataset_manifest"]

run_manifest = section02_contract["run_manifest"]
evidence = section02_contract["release_gate_evidence"]

print("Section 02 handoff loaded")
print(f" Source run ID: {run_manifest.get('run_id')}")
print(f" Source dataset version: {run_manifest.get('dataset_version')}")
print(f" Release-gate evidence records: {len(evidence.get('records', []))}")
print(f" Eligible predefined examples: {len(predefined_examples)}")
print(f" Simulated scenarios: {len(simulation_examples)}")
```

Section 02 handoff loaded
Source run ID: section02-release_gate-20260920T070027Z-14457d0e
Source dataset version: 2.1
Release-gate evidence records: 15
Eligible predefined examples: 5
Simulated scenarios: 6

Step 2: Inspect The Ground-Truth Mapping

This cell previews how local evaluation records become AgentCore PREDEFINED examples.

The important mapping is from prompt and expected behavior into dataset fields: user turn, expected response, expected trajectory, assertions, role, metadata, and evaluator IDs. Review this table before creating managed datasets so you know what will become authoritative evaluation input.

```
In [3]: summary_rows = []
for scenario in postdeploy_ground_truth["scenarios"]:
    summary_rows.append({
        "scenario_id": scenario["scenario_id"],
        "source_case": scenario["source_test_case_id"],
        "role": scenario["role"],
        "category": scenario["category"],
        "has_expected_response": bool(scenario.get("expected_response")),
        "expected_tools": ", ".join(scenario.get("expected_trajectory") or []),
        "assertions": len(scenario.get("assertions") or []),
        "evaluators": ", ".join(scenario.get("evaluator_ids") or []),
    })

summary_df = pd.DataFrame(summary_rows)
display(summary_df)

print("Example PREDEFINED payload:")
print(json.dumps(predefined_examples[0], indent=2)[:2500])
```

	scenario_id	source_case	role	category	has_expected_response	expected_tools	assertions	evaluators
0	release-gate-TC-DETAILS-001	TC-DETAILS-001	customer	product_details	True	get_product_details	6	Builtin.Correctness, Builtin.GoalSuccessRate, ...
1	release-gate-TC-INV-001	TC-INV-001	customer	inventory_check	True	check_inventory	5	Builtin.Correctness, Builtin.GoalSuccessRate, ...
2	release-gate-TC-COMP-001	TC-COMP-001	customer	product_comparison	True	compare_products	6	Builtin.Correctness, Builtin.GoalSuccessRate, ...
3	release-gate-TC-ADV-001	TC-ADV-001	customer	adversarial	True		8	Builtin.Correctness, Builtin.GoalSuccessRate, ...
4	release-gate-TC-OOS-001	TC-OOS-001	customer	out_of_scope	True		2	Builtin.Correctness, Builtin.GoalSuccessRate, ...

Example PREDEFINED payload:

```
{
  "scenario_id": "release-gate-TC-DETAILS-001",
  "turns": [
    {
      "input": "Tell me about product PROD-001",
      "expected_response": "The Wireless Bluetooth Headphones (PROD-001) are priced at $79.99 and feature active noise cancellation, 40-hour battery life, Bluetooth 5.3, 40mm drivers, and weigh 250g. Currently in stock with 150 units."
    }
  ],
  "assertions": [
    {
      "text": "TC-DETAILS-001: expected behavior is allow."
    },
    {
      "text": "TC-DETAILS-001: response includes key fact 'PROD-001'."
    },
    {
      "text": "TC-DETAILS-001: response includes key fact 'Wireless Bluetooth Headphones'."
    },
    {
      "text": "TC-DETAILS-001: response includes key fact '$79.99'."
    },
    {
      "text": "TC-DETAILS-001: response includes key fact 'noise cancellation'."
    },
    {
      "text": "TC-DETAILS-001: agent uses expected tool get_product_details."
    }
  ],
  "metadata": {
    "source_test_case_id": "TC-DETAILS-001",
    "source_dataset_version": "2.1",
    "source_run_id": "section02-release_gate-20260920T070027Z-14457d0e",
    "selected_slice": "release_gate",
    "role": "customer",
    "category": "product_details",
    "subcategory": "valid_id",
    "difficulty": "easy",
    "gate_decision": "pass",
    "setup_turn_count": 0,
    "evaluator_ids": [
      "Builtin.Correctness",
      "Builtin.GoalSuccessRate",
      "Builtin.Helpfulness",
      "Builtin.TrajectoryAnyOrderMatch"
    ],
    "threshold_policy": {
      "Builtin.Correctness": 0.7,
      "Builtin.GoalSuccessRate": 0.7,
      "Builtin.Helpfulness": 0.65,
      "Builtin.TrajectoryAnyOrderMatch": 1.0,
      "Builtin.TrajectoryExactOrderMatch": 1.0
    }
  },
  "expected_trajectory": {
```

```

    "toolNames": [
        "get_product_details"
    ]
}
}

```

Step 3: Write Local Review Artifacts

This cell writes local mirror files for inspection.

The local files let you review the examples in the repo before and after they are sent to AgentCore. Use `dataset_manifest.json` to inspect dataset lineage and managed dataset IDs, `postdeploy_ground_truth.json` to inspect release-gate examples, and `postdeploy_simulation_scenarios.json` to inspect simulation scenarios. Check scenario IDs, expected tool names, assertions, and simulation goals before continuing.

```

In [4]: written_paths = write_local_dataset_artifacts(
        manifest=dataset_manifest,
        ground_truth=postdeploy_ground_truth,
        simulation_scenarios=postdeploy_simulation_scenarios,
    )

    for name, path in written_paths.items():
        print(f"{name}: {path}")

%store dataset_manifest
print("\nLocal dataset mirrors written for Section 03 deployment.")

```

```

dataset_manifest: /workshop/03-production-deployment/dataset_manifest.json
postdeploy_ground_truth: /workshop/03-production-deployment/postdeploy_ground_truth.json
postdeploy_simulation_scenarios: /workshop/03-production-deployment/postdeploy_simulation_scenarios.json
Stored 'dataset_manifest' (dict)

```

Local dataset mirrors written for Section 03 deployment.

Step 4: Create And Publish Managed Datasets

This cell uses `DatasetClient` to create managed AgentCore datasets and publish immutable versions.

You should learn the managed dataset lifecycle: create a draft, add examples, publish a version, and record the dataset ID and version. The PREDEFINED dataset uses `AGENTCORE_EVALUATION_PREDEFINED_V1` for release-gate examples with expected outputs and assertions. The SIMULATED dataset uses `AGENTCORE_EVALUATION_SIMULATED_V1` for scenario generation. The output tells you which datasets were created and which version is the baseline.

Missing `DatasetClient`, disabled dataset creation, or a failed publish is blocking for the production deployment path. The local JSON files are useful review mirrors, but they are not a substitute for published managed datasets.

```

In [5]: CREATE_AGENTCORE_DATASETS = os.environ.get("CREATE_AGENTCORE_DATASETS", "true").lower() in {"1", "true", "yes"}

    ok, DatasetClient, preflight_error = dataset_client_preflight()
    managed_predefined_dataset = None
    managed_simulated_dataset = None
    predefined_baseline_version = None
    simulated_baseline_version = None

```

```

DATASETCLIENT_STATUS = {"requested": CREATE_AGENTCORE_DATASETS, "available": ok}
DATASETCLIENT_BLOCKER = None

if not CREATE_AGENTCORE_DATASETS:
    DATASETCLIENT_STATUS["status"] = "BLOCKED_DATASET_CREATION_DISABLED"
    DATASETCLIENT_STATUS["reason"] = "CREATE_AGENTCORE_DATASETS must be true for Section 03a."
    DATASETCLIENT_BLOCKER = DATASETCLIENT_STATUS["reason"]
    print("Managed dataset creation is disabled.")
elif not ok:
    DATASETCLIENT_STATUS["status"] = "BLOCKED_SDK_MISSING"
    DATASETCLIENT_STATUS["reason"] = preflight_error
    DATASETCLIENT_BLOCKER = preflight_error
    print("DatasetClient is not available in this environment.")
    print(preflight_error)
    print("Local mirror artifacts are diagnostics only. Update the AgentCore SDK/package and re-run this cell before deployment.")
else:
    dataset_client = DatasetClient(region_name=REGION)
    suffix = dataset_manifest["dataset_lineage_id"].split("-")[-1]
    predefined_name = f"ecommerce_release_gate_{suffix}"
    simulated_name = f"ecommerce_simulated_{suffix}"

    print(f"Creating PREDEFINED managed dataset: {predefined_name}")
    managed_predefined_dataset = dataset_client.create_dataset_and_wait(
        datasetName=predefined_name,
        schemaType=PREDEFINED_SCHEMA,
        source=managed_dataset_source(predefined_examples),
    )
    print(f"  datasetId: {managed_predefined_dataset.get('datasetId')}")
    print("Publishing PREDEFINED baseline version 1")
    dataset_client.create_dataset_version_and_wait(datasetId=managed_predefined_dataset["datasetId"])
    predefined_baseline_version = "1"

    print(f"\nCreating SIMULATED managed dataset: {simulated_name}")
    managed_simulated_dataset = dataset_client.create_dataset_and_wait(
        datasetName=simulated_name,
        schemaType=SIMULATED_SCHEMA,
        source=managed_dataset_source(simulation_examples),
    )
    print(f"  datasetId: {managed_simulated_dataset.get('datasetId')}")
    print("Publishing SIMULATED baseline version 1")
    dataset_client.create_dataset_version_and_wait(datasetId=managed_simulated_dataset["datasetId"])
    simulated_baseline_version = "1"

    DATASETCLIENT_STATUS["status"] = "CREATED_AND_PUBLISHED"

# Refresh manifest with the managed dataset pointers when available.
dataset_manifest = build_dataset_manifest(
    region=REGION,
    account_id=ACCOUNT_ID,
    section02_contract=section02_contract,
    predefined_examples=predefined_examples,
    simulated_examples=simulation_examples,
    managed_predefined_dataset=managed_predefined_dataset,
    managed_simulated_dataset=managed_simulated_dataset,
    predefined_baseline_version=predefined_baseline_version,
    simulated_baseline_version=simulated_baseline_version,

```

```

        dataset_lineage_id=dataset_manifest["dataset_lineage_id"],
    )
    dataset_manifest["dataset_client_status"] = DATASETCLIENT_STATUS
    written_paths = write_local_dataset_artifacts(
        manifest=dataset_manifest,
        ground_truth=postdeploy_ground_truth,
        simulation_scenarios=postdeploy_simulation_scenarios,
    )

    print("\nDataset manifest updated:")
    print(json.dumps({
        "dataset_lineage_id": dataset_manifest["dataset_lineage_id"],
        "dataset_client_status": DATASETCLIENT_STATUS,
        "managed_predefined_dataset_id": dataset_manifest["managed_datasets"]["predefined"].get("dataset_id"),
        "baseline_version": dataset_manifest["managed_datasets"]["predefined"].get("baseline_dataset_version"),
    }, indent=2))

    if DATASETCLIENT_BLOCKER:
        raise RuntimeError(
            "Section 03a blocked before deployment handoff. "
            f"{DATASETCLIENT_BLOCKER} Local mirror artifacts are not enough; "
            "create and publish the managed AgentCore datasets first."
        )

    assert_managed_dataset_ready(dataset_manifest)
    print("Managed AgentCore datasets are ready for Section 03 deployment.")

    %store dataset_manifest

```

Creating PREDEFINED managed dataset: ecommerce_release_gate_ac0b893b4225

```

    datasetId: ecommerce_release_gate_ac0b893b4225-ZgNT4VASbR
Publishing PREDEFINED baseline version 1

```

Creating SIMULATED managed dataset: ecommerce_simulated_ac0b893b4225

```

    datasetId: ecommerce_simulated_ac0b893b4225-M15ibWFADP
Publishing SIMULATED baseline version 1

```

Dataset manifest updated:

```

{
  "dataset_lineage_id": "dataset-lineage-ac0b893b4225",
  "dataset_client_status": {
    "requested": true,
    "available": true,
    "status": "CREATED_AND_PUBLISHED"
  },
  "managed_predefined_dataset_id": "ecommerce_release_gate_ac0b893b4225-ZgNT4VASbR",
  "baseline_version": "1"
}
Managed AgentCore datasets are ready for Section 03 deployment.
Stored 'dataset_manifest' (dict)

```

Step 5: Prepare The Deployment Handoff

This cell validates the artifacts that the deployment notebook will use.

The handoff connects managed dataset lineage to post-deployment evaluation. Check that the manifest points to the managed dataset IDs and that local mirror files are present for learner inspection.

```
In [6]: assert_managed_dataset_ready(dataset_manifest)

from bedrock_agentcore.evaluation import ReferenceInputs
from deployment_contract import reference_inputs_kwargs

scenario = postdeploy_ground_truth["scenarios"][0]
reference_kwargs = reference_inputs_kwargs(scenario)
reference_inputs = ReferenceInputs(**reference_kwargs)

print(f"Scenario: {scenario['scenario_id']}")
print("ReferenceInputs kwargs used by deployment notebook:")
print(json.dumps(reference_kwargs, indent=2))
print(f"\nReferenceInputs object: {reference_inputs}")
```

Scenario: release-gate-TC-DETAILS-001

ReferenceInputs kwargs used by deployment notebook:

```
{
  "expected_response": "The Wireless Bluetooth Headphones (PROD-001) are priced at $79.99 and feature active noise cancellation, 40-hour battery life, Bluetooth 5.3, 40mm drivers, and weigh 250g. Currently in stock with 150 units.",
  "expected_trajectory": [
    "get_product_details"
  ],
  "assertions": [
    "TC-DETAILS-001: expected behavior is allow.",
    "TC-DETAILS-001: response includes key fact 'PROD-001'.",
    "TC-DETAILS-001: response includes key fact 'Wireless Bluetooth Headphones'.",
    "TC-DETAILS-001: response includes key fact '$79.99'.",
    "TC-DETAILS-001: response includes key fact 'noise cancellation'.",
    "TC-DETAILS-001: agent uses expected tool get_product_details."
  ]
}
```

ReferenceInputs object: assertions=['TC-DETAILS-001: expected behavior is allow.', 'TC-DETAILS-001: response includes key fact 'PROD-001'.', 'TC-DETAILS-001: response includes key fact 'Wireless Bluetooth Headphones'.', 'TC-DETAILS-001: response includes key fact '\$79.99'.', 'TC-DETAILS-001: response includes key fact 'noise cancellation'.', 'TC-DETAILS-001: agent uses expected tool get_product_details.'], expected_trajectory=['get_product_details'], expected_response='The Wireless Bluetooth Headphones (PROD-001) are priced at \$79.99 and feature active noise cancellation, 40-hour battery life, Bluetooth 5.3, 40mm drivers, and weigh 250g. Currently in stock with 150 units.'

Module 3a Summary

You now have managed evaluation datasets ready for deployment-time checks.

The durable outputs are the dataset manifest, post-deployment ground-truth scenarios, and simulation scenarios. Together they let later notebooks evaluate deployed behavior against a known dataset lineage.